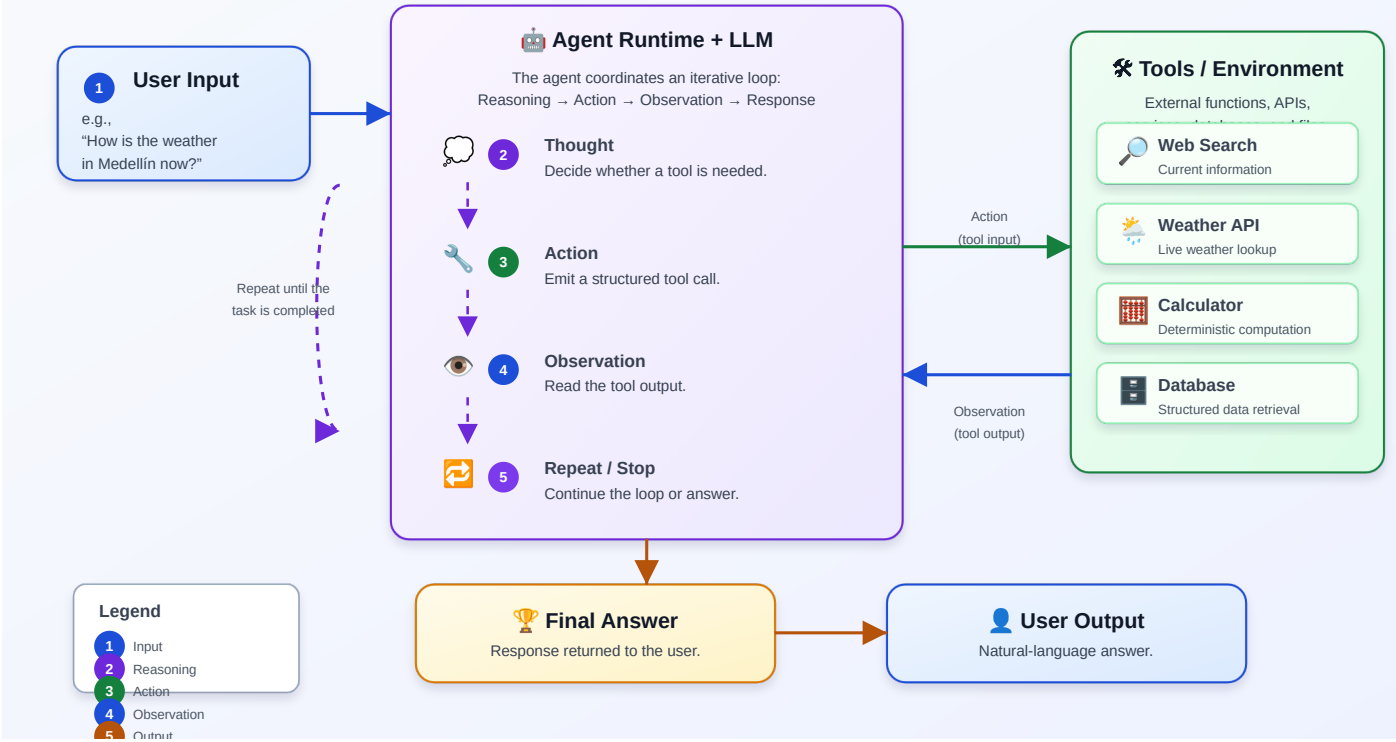
SI3003 - Inteligencia Artificial

CLASE 12 — AGENTES CON LLMS: DEL LOOP AGÉNTICO A LANGGRAPH

Agent from Scratch: Architecture

From prompt to action through tools, observations, and a final answer



Agenda

1. De LLM a agente
2. El loop agéntico desde cero
3. Razonamiento: CoT, ReAct y ToT
4. Herramientas
5. LangGraph: el mismo loop como

EAFIT

UNIVERSIDAD

Parte 1 — De LLM a agente

Repaso: el agente de la Clase 1

Un agente es una función que mapea la **secuencia de percepciones** a una acción:

$$f : \mathcal{P}^* \to \mathcal{A}$$

El programa del agente corre en un ciclo: percibir → decidir → actuar → percibir el efecto.

Lo que ya teníamos:

- Agentes reflejos, basados en modelo, en metas, en utilidad.
- MDPs y RL: la política $\pi(a \mid s)$ se *calcula* o se *aprende*.

Lo nuevo hoy:

- La política es un **LLM preentrenado**.
- La percepción y la acción son **texto** (mensajes y JSON).
- No entrenamos nada: el comportamiento se programa con el *prompt* y las herramientas.

¿Qué es un agente (con LLM)?

Una aplicación autónoma que persigue un objetivo **observando** el entorno y **actuando** sobre él mediante herramientas.

Combina tres cosas:

- **Razonamiento**: el LLM decide qué hacer a continuación.
- **Acción**: llamadas a herramientas (APIs, bases de datos, código).
- **Información externa**: lo que devuelven esas herramientas entra al siguiente paso.

Puede planear y ejecutar tareas de varios pasos **sin una instrucción humana en cada paso**.

Wiesinger, Marlow & Vuskovic, *Agents* (Google, 2024).

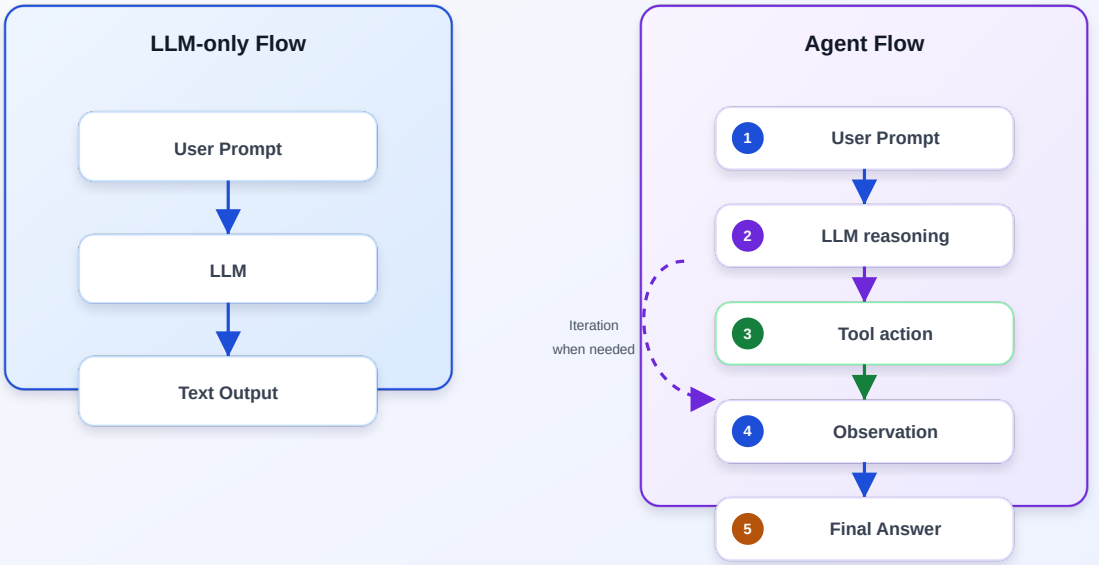
Modelos vs. agentes

	Modelo (LLM)	Agente
Conocimiento	Limitado a los datos de entrenamiento	Ampliado con herramientas externas
Inferencia	Una sola predicción por consulta	Varias inferencias encadenadas, decididas por la capa de orquestación
Historial	No lo gestiona (es *stateless*)	Gestiona el historial de la sesión
Herramientas	No tiene implementación nativa	Integradas en la arquitectura
Lógica	La pone el usuario en el *prompt* (CoT, ReAct)	Arquitectura cognitiva nativa (CoT, ReAct, frameworks)

Adaptado de Wiesinger et al. (2024).

LLM vs Agent

Text generation versus action-oriented problem solving



Formalización: el LLM como política

Un LLM define una distribución autoregresiva sobre secuencias de tokens:

$$p_\theta(y \mid x) = \prod_{k=1}^{|y|} p_\theta(y_k \mid x, y_{<k})$$

El agente mantiene un **historial** h_t (el estado) y en cada paso:

$$\begin{aligned} a_t &\sim \pi_\theta(\cdot \mid h_t) \equiv p_\theta(\cdot \mid \text{prompt}(h_t)) && (\text{el LLM elige una acción}) \\ o_t &= E(a_t) && (\text{el entorno / herramienta responde}) \\ h_{t+1} &= h_t \oplus a_t \oplus o_t && (\text{concatenación al historial}) \end{aligned}$$

con $h_0 = (\text{system}, \text{user})$. Termina cuando a_t es una **respuesta final** o cuando $t = T_{\max}$.

El espacio de acciones

El LLM emite texto, pero la orquestación lo interpreta en dos tipos de acción:

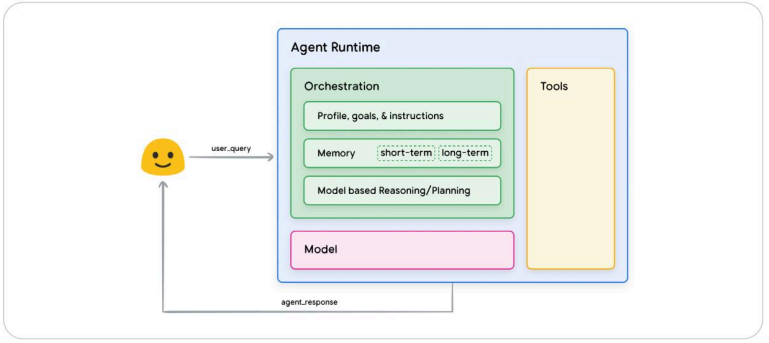
$$\mathcal{A} = \underbrace{\mathcal{A}_{\text{tool}}}_{\text{llamar herramienta}} \cup \underbrace{\mathcal{A}_{\text{final}}}_{\text{responder al usuario}}$$

- $a_t \in \mathcal{A}_{\text{tool}}$: un objeto estructurado $(\texttt{name}, \texttt{args})$ → se ejecuta y se observa o_t.
- $a_t \in \mathcal{A}_{\text{final}}$: texto libre → el episodio termina.

En la API (formato OpenAI) esa distinción llega como `finish_reason`:

`finish_reason`	Acción	Qué hace el runtime
`"tool_calls"`	$a_t \in \mathcal{A}_{\text{tool}}$	parsea, ejecuta, agrega `role: tool`
`"stop"`	$a_t \in \mathcal{A}_{\text{final}}$	devuelve el texto

Arquitectura cognitiva



(1) Modelo: el LLM que toma las decisiones. Puede ser general, multimodal o ajustado (*fine-tuned*).

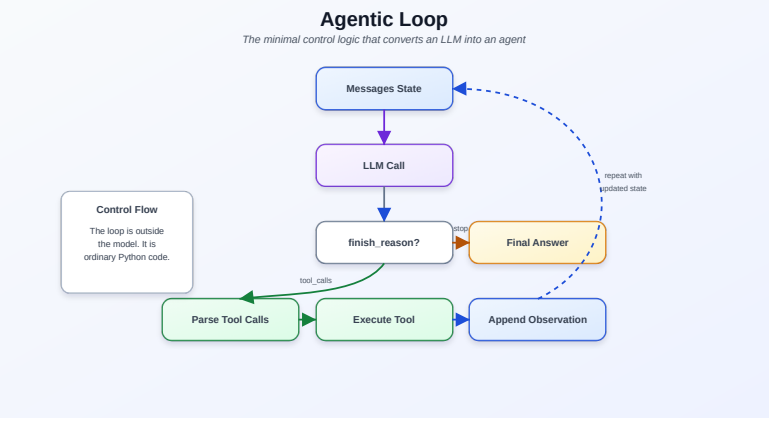
(2) Herramientas: conectan el agente con el mundo exterior — extensiones, funciones y almacenes de datos.

(3) Orquestación: el ciclo que recibe información, razona, decide y actúa; gestiona memoria (corto y largo plazo), estado y planificación.

Figura: Wiesinger et al., *Agents* (Google, 2024).

Parte 2 — El loop agéntico desde cero

El loop agéntico




```
function AGENTE(user, tools, T_max)
  h ← [system, user]
  for t = 1 to T_max do
    m ← LLM(h, tools)
    if m no tiene tool_calls then
      return m.content       # respuesta final
    h ← h ⊕ m                # registrar la intención
    for each (id, name, args) in m.tool_calls do
      o ← REGISTRY[name](args)
      h ← h ⊕ (tool, id, o)  # registrar la observación
  return FALLO("se agotaron las iteraciones")
```

El mismo loop en Python

```python
def run_agent(user_message, tools, registry, system, max_iter=10):
    messages = [{"role": "system", "content": system},
                {"role": "user",   "content": user_message}]
    for it in range(max_iter):
        resp = client.chat.completions.create(
            model=MODEL, messages=messages, tools=tools, tool_choice="auto")
        msg = resp.choices[0].message

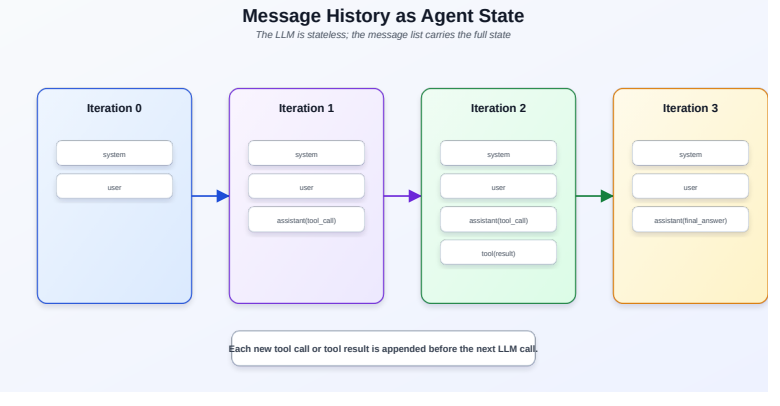
        if not msg.tool_calls:                    # a_t ∈ A_final
            return msg.content

        messages.append(msg)                      # h ← h ⊕ a_t
        for tc in msg.tool_calls:                 # puede haber varias
            fn   = registry.get(tc.function.name)
            args = json.loads(tc.function.arguments)
            try:
                obs = fn(**args) if fn else f"Error: herramienta {tc.function.name} no existe"
            except Exception as e:
                obs = f"Error: {e}"               # el error también es observación
            messages.append({"role": "tool", "tool_call_id": tc.id,
                             "content": str(obs)})  # h ← h ⊕ o_t
    raise RuntimeError("max_iter alcanzado")
```

El historial de mensajes *es* el estado

El LLM es **stateless**: no recuerda nada entre llamadas. Toda la memoria del agente es la lista `messages` que se reenvía completa en cada iteración.

Iteración	`messages`
0	`[system, user]`
1	`[…, assistant(tool_call)]`
2	`[…, tool(result)]`
3	`[…, assistant(final)]`



Consecuencia: lo que no esté en `messages`, el agente no lo sabe.

El costo del loop crece cuadráticamente

Sea L_0 la longitud (en tokens) del prompt inicial y $\Delta_k = |a_k| + |o_k|$ lo que agrega la iteración k. La llamada t procesa todo el historial:

$$L_t = L_0 + \sum_{k=0}^{t-1} \Delta_k$$

El costo total de un episodio de T llamadas es

$$C(T) = \sum_{t=0}^{T-1} L_t = T L_0 + \sum_{k=0}^{T-2} (T-1-k)\,\Delta_k \overset{\Delta_k = \Delta}{=} T L_0 + \Delta\,\frac{T(T-1)}{2} = O(T^2)$$

Ejemplo: $L_0 = 1000$, $\Delta = 1500$ (una búsqueda web), $T = 10 \to C = 10\,000 + 1500 \cdot 45 = 77\,500$ tokens de entrada.

Además el historial no puede superar la ventana de contexto del modelo (Clase 9): es un límite duro, no solo un costo.

EAFIT

UNIVERSIDAD

Parte 3 — Razonamiento: CoT, ReAct y ToT

Chain-of-Thought (CoT)

Pedirle al modelo que escriba pasos intermedios z antes de la respuesta y. La respuesta correcta es la que **marginaliza** sobre los razonamientos:

$$p_\theta(y \mid x) = \sum_z p_\theta(y \mid x, z)\, p_\theta(z \mid x)$$

- **CoT greedy**: genera un solo z y responde condicionado a él — una muestra de la suma.
- **Self-consistency**: muestrea N razonamientos $z^{(i)} \sim p_\theta(z \mid x)$ con temperatura > 0, obtiene $y^{(i)}$ y vota:

$$\hat{y} = \arg\max_y \sum_{i=1}^{N} \mathbf{1}\left[y^{(i)} = y\right] \;\approx\; \arg\max_y p_\theta(y \mid x)$$

Wei et al., *Chain-of-Thought Prompting Elicits Reasoning in LLMs* (2022); Wang et al., *Self-Consistency Improves CoT Reasoning* (2023).

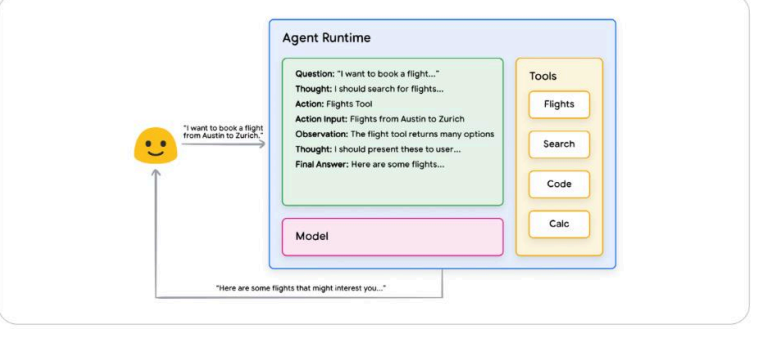
ReAct: razonar y actuar

ReAct intercala pensamientos en lenguaje natural con acciones sobre el entorno. Amplía el espacio de acciones con el espacio del lenguaje $\mathcal{L}$:

$$\hat{\mathcal{A}} = \mathcal{A} \cup \mathcal{L}, \qquad o_t = \begin{cases} E(a_t) & a_t \in \mathcal{A} \\ \varnothing & a_t \in \mathcal{L} \ (\text{un pensamiento no cambia el entorno}) \end{cases}$$

Traza típica:

```
Question:  vuelo de Austin a Zúrich
Thought:   debo buscar vuelos
Action:    flights_tool
Action Input: Austin → Zurich
Observation: [lista de vuelos]
Thought:   debo presentarlos
Final Answer: estos son los vuelos…
```



Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models* (ICLR 2023). Figura: Wiesinger et al. (2024).

Tree-of-Thoughts (ToT): razonar es buscar

Cada "pensamiento" parcial es un **estado** en un árbol de búsqueda (Clase 2):

- **Generador** $G(s) \to \{s'_1, \ldots, s'_k\}$: el LLM propone k continuaciones.
- **Evaluador** $V(s) \in \mathbb{R}$: el LLM (u otra función) puntúa qué tan prometedor es s — una heurística, como $h(n)$ en A*.
- **Búsqueda**: BFS con haz de ancho b (beam search, Clase 3) o DFS con poda.

```
function TOT-BFS(x, G, V, b, D)
  S ← {x}
  for d = 1 to D do
    S' ← ∪_{s ∈ S} G(s)                 # expandir
    S  ← los b estados de S' con mayor V  # podar
  return argmax_{s ∈ S} V(s)
```

Comparación de técnicas de razonamiento

Técnica	Estructura	Usa el entorno	Llamadas al LLM
CoT	Una cadena $z \to y$	No	1
Self-consistency	N cadenas + votación	No	N
ReAct	Cadena intercalada con acciones	Sí	T (iteraciones del loop)
ToT	Árbol + búsqueda + heurística	Opcional	$O(b\,k\,D)$

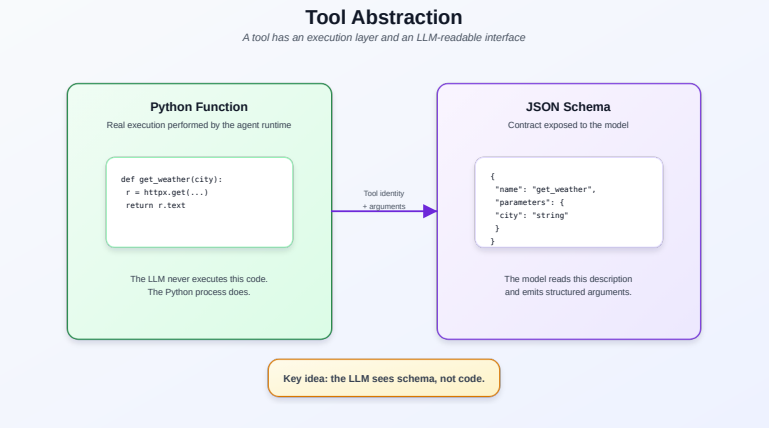
Regla práctica: si la información necesaria está fuera del modelo, se necesita ReAct (herramientas). Si está dentro pero el razonamiento es difícil, CoT / self-consistency / ToT.

EAFIT

UNIVERSIDAD

Parte 4 — Herramientas

Las dos caras de una herramienta




```python
TOOLS = [{
  "type": "function",
  "function": {
    "name": "get_weather",
    "description": "Clima actual de una ciudad.",
    "parameters": {
      "type": "object",
      "properties": {
        "city": {"type": "string",
                 "description": "Ej: 'Medellín'"}},
      "required": ["city"]}}}]

def get_weather(city: str) -> str:
    url = f"https://wttr.in/{city}?format=3"
    return httpx.get(url).text

REGISTRY = {"get_weather": get_weather}
```

El LLM solo ve el **esquema**. Nunca ve ni ejecuta el código.

Una tool call es generación estructurada

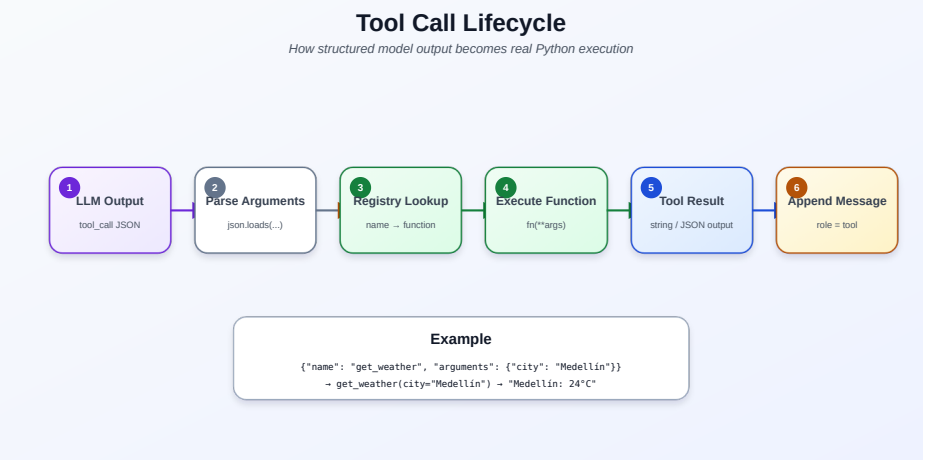
El LLM genera la llamada token a token, como cualquier texto. La probabilidad de una llamada se factoriza en nombre y argumentos:

$$p_\theta(\texttt{name}, \texttt{args} \mid h_t) = p_\theta(\texttt{name} \mid h_t) \prod_k p_\theta\big(\texttt{args}_k \mid h_t, \texttt{name}, \texttt{args}_{<k}\big)$$

Consecuencias de ingeniería:

- Nada garantiza que `args` sea JSON válido ni que cumpla el esquema → **validar** antes de ejecutar (`json.loads`, tipos, campos `required`).
- El nombre puede no existir en `REGISTRY` → devolver el error como observación.
- Muchos proveedores aplican **decodificación restringida**: en cada paso ponen $p_\theta(y_k \mid \cdot) = 0$ para los tokens que harían el JSON inválido según el esquema.

Ciclo de vida de una tool call



1. El LLM emite `tool_calls` (JSON).
2. Parsear: `json.loads(arguments)`.
3. Resolver: `REGISTRY[name]` → `fn`.
4. Ejecutar: `fn(**args)`.
5. Obtener el resultado (string).
6. Agregar `{"role": "tool", "tool_call_id": id, "content": …}`.

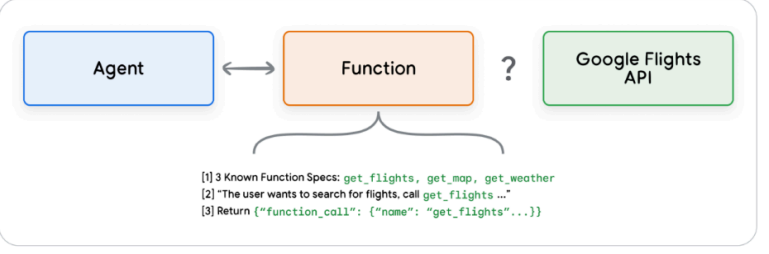
```
LLM emite:      {"name": "get_weather", "arguments": "{\"city\": \"Medellín\"}"}
Runtime:        get_weather(city="Medellín")  →  "Medellín: 24°C, parcialmente nublado"
Historial:      {"role": "tool", "tool_call_id": "call_01", "content": "Medellín: 24°C, …"}
```

Tres tipos de herramientas

Extensiones — el **agente** llama directamente a la API externa. La extensión enseña al modelo cómo usarla (con ejemplos).



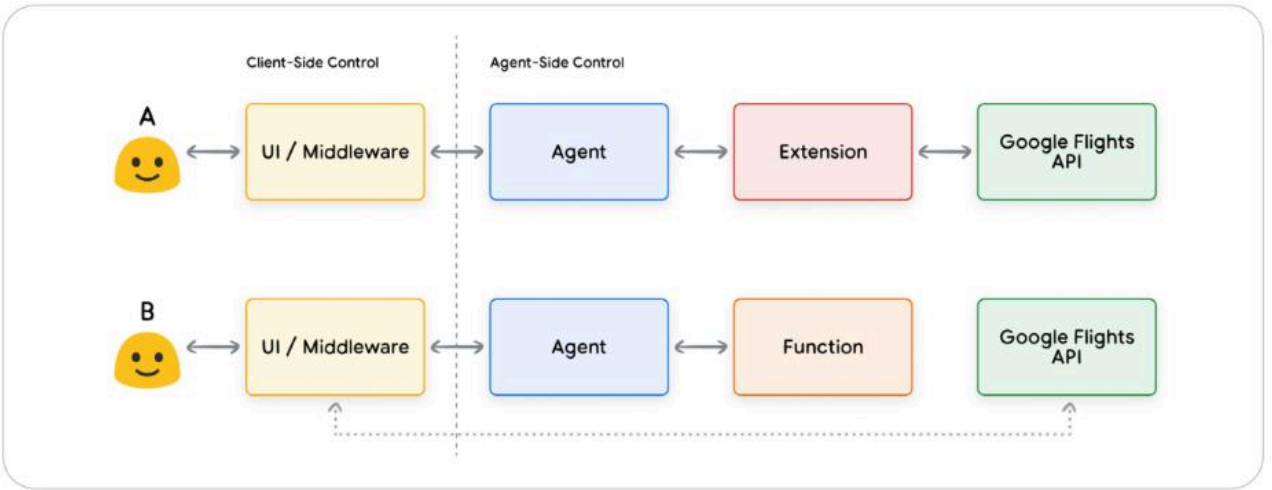
Funciones — el modelo solo **emite** la llamada (nombre + argumentos); la ejecución ocurre en el **cliente**.



Almacenes de datos — acceso a información (documentos, web, tablas) vía embeddings: es RAG (Clase 10) usado como herramienta.

Figuras: Wiesinger et al., *Agents* (Google, 2024).

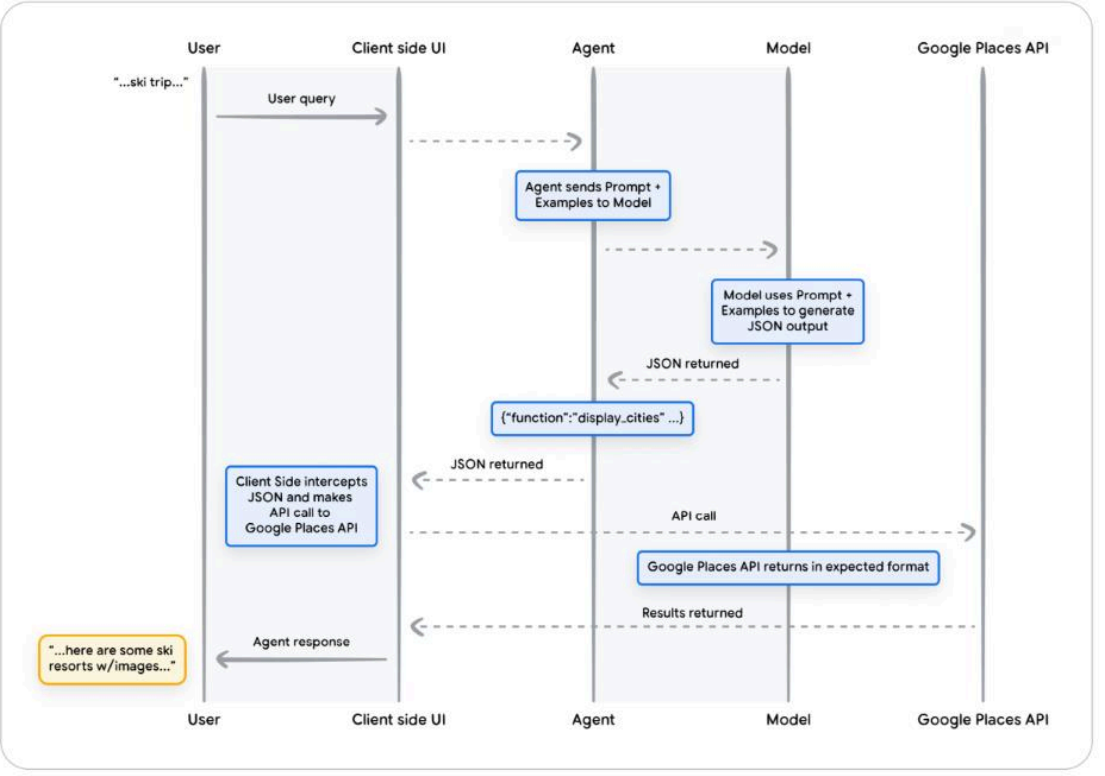
Extensión vs. función: ¿quién tiene el control?



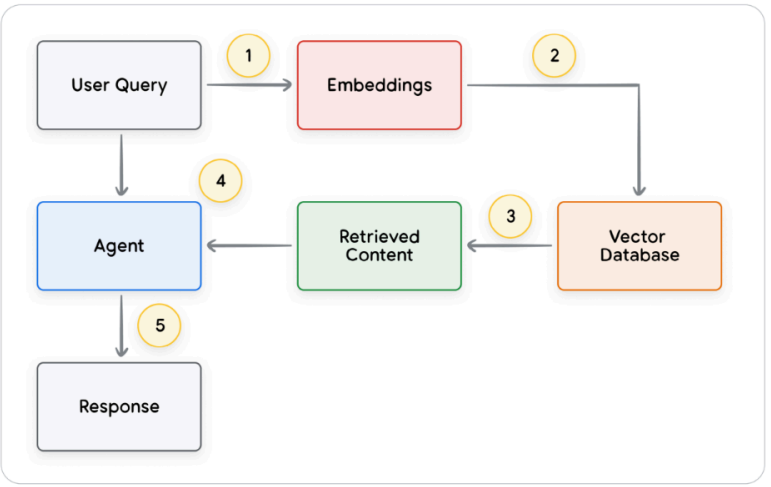
A — Extensión (control del agente): menos código en el cliente, pero el agente necesita credenciales y acceso de red.

B — Función (control del cliente): el cliente valida, filtra, pide confirmación humana o ejecuta en otro sistema antes de llamar la API.

Function calling paso a paso



Almacenes de datos = RAG como herramienta



(1) La consulta se convierte en embedding $\mathbf{q} = E(\text{query})$.

(2) Se buscan los k chunks más similares:

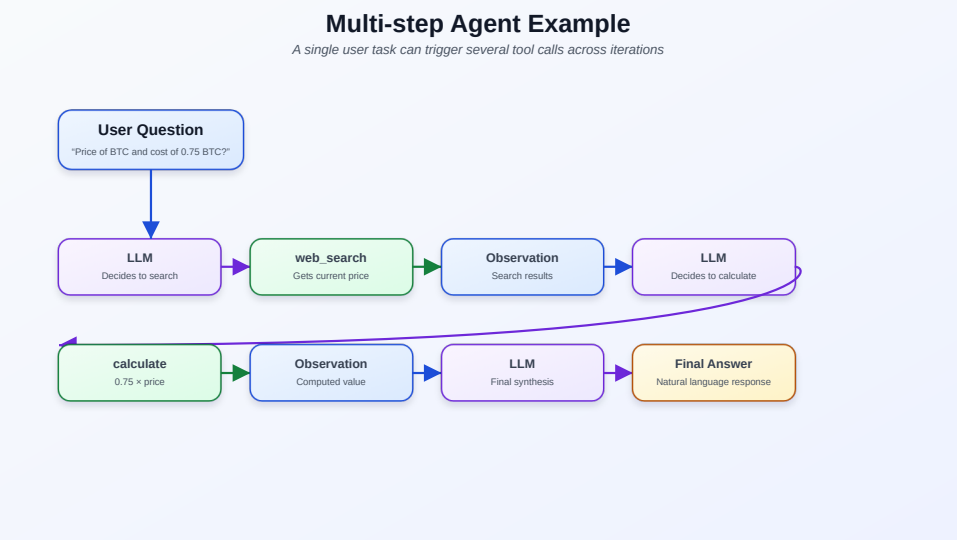
$$\mathcal{R}_k(\mathbf{q}) = \underset{c \in \mathcal{C},\ |\cdot|=k}{\text{top-}k} \frac{\mathbf{q}^\top \mathbf{e}_c}{\|\mathbf{q}\|\,\|\mathbf{e}_c\|}$$

(3) El contenido recuperado vuelve al agente como **observación** o_t.

La diferencia con la Clase 10: aquí el **agente decide** cuándo buscar y qué consulta escribir.

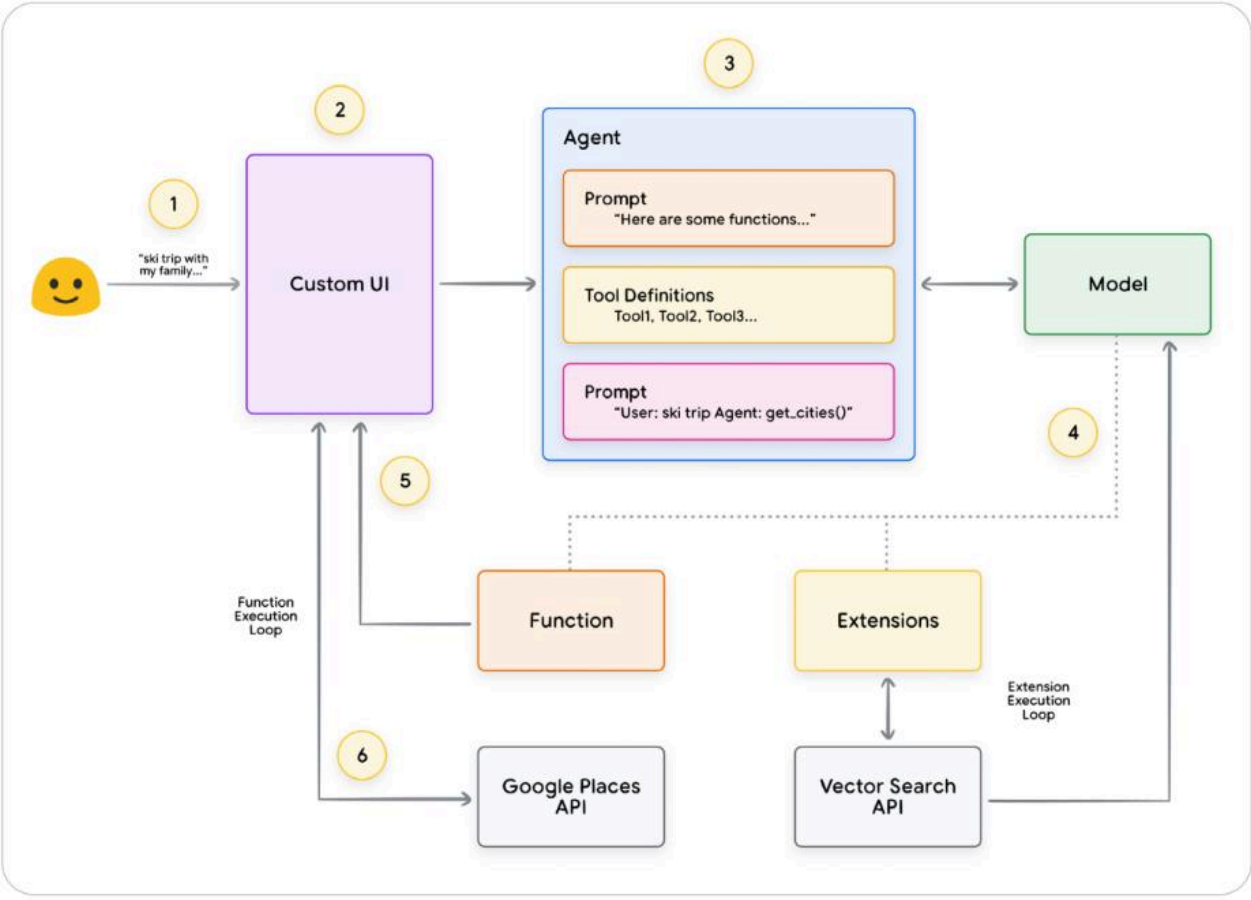
Figura: Wiesinger et al. (2024).

Ejemplo multi-paso: encadenar herramientas



Pregunta: "¿Cuánto cuestan 0.75 BTC hoy?" $\rightarrow$ `web_search("precio BTC")` $\rightarrow$ `calculate("0.75 * 94200")` $\rightarrow$ respuesta.

El agente no planea todo al inicio: en cada iteración decide el siguiente paso mirando h_t.

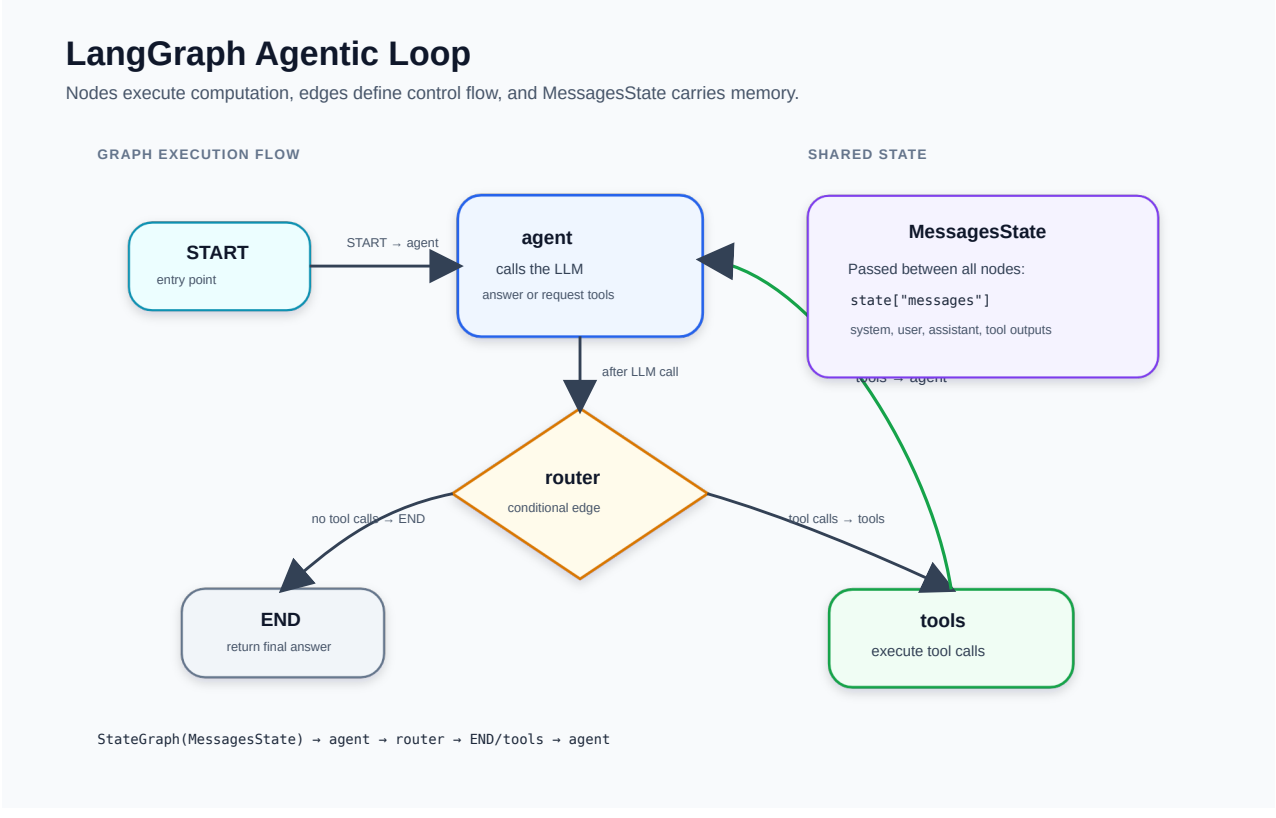


Arquitectura general: UI, agente (prompt + definiciones de herramientas), modelo, funciones y extensiones. Wiesinger et al. (2024).

Parte 5 — LangGraph: el mismo loop como grafo

¿Por qué un framework?

En el loop manual, el control de flujo **está en el código** (`for`, `if`, `append`). En LangGraph, el control de flujo **es la estructura de un grafo**.



Formalización: un grafo de estado

Un programa LangGraph es una tupla $(V, E, S, \oplus)$:

- S: el **estado** (un `TypedDict`). Para un agente: $S = \{\texttt{messages}\}$.
- V: **nodos**, funciones $f_v : S \to \Delta S$ que devuelven una **actualización parcial**.
- $\oplus$: el **reducer** que combina estado y actualización: $S \leftarrow S \oplus f_v(S)$.
- E: aristas fijas $u \to v$ y aristas **condicionales** con un router $r : S \to V \cup \{\texttt{END}\}$.

Para `MessagesState`, el reducer es concatenación (`add_messages`):

$$S.\texttt{messages} \leftarrow S.\texttt{messages} \oplus [m_1, \ldots, m_j]$$

Es exactamente $h_{t+1} = h_t \oplus a_t \oplus o_t$ de la Parte 1 — ahora lo hace el framework.

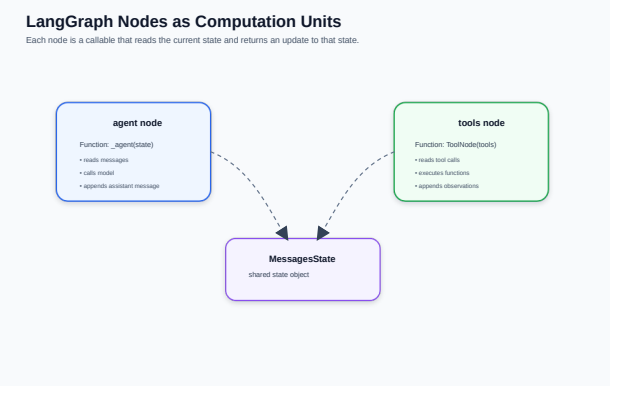
Nodos como unidades de cómputo

```python
@tool
def web_search(query: str) -> str:
    """Busca información actualizada en la web."""
    ...

TOOLS = [web_search, get_weather, calculate]
llm = ChatOpenAI(model=MODEL).bind_tools(TOOLS)

def agent_node(state: MessagesState) -> dict:
    sys = SystemMessage(SYSTEM_PROMPT)
    response = llm.invoke([sys] + state["messages"])
    return {"messages": [response]}  # parcial

tool_node = ToolNode(TOOLS)  # parsea, ejecuta, empaqueta
```



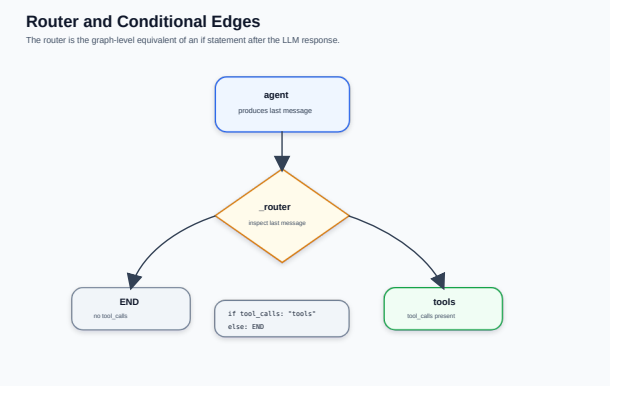
- `agent`: lee `messages`, devuelve `[AIMessage]`.
- `tools`: lee el último `AIMessage`, devuelve `[ToolMessage, …]`.

Router y construcción del grafo

```python
def router(state: MessagesState):
    last = state["messages"][-1]
    return "tools" if last.tool_calls else END

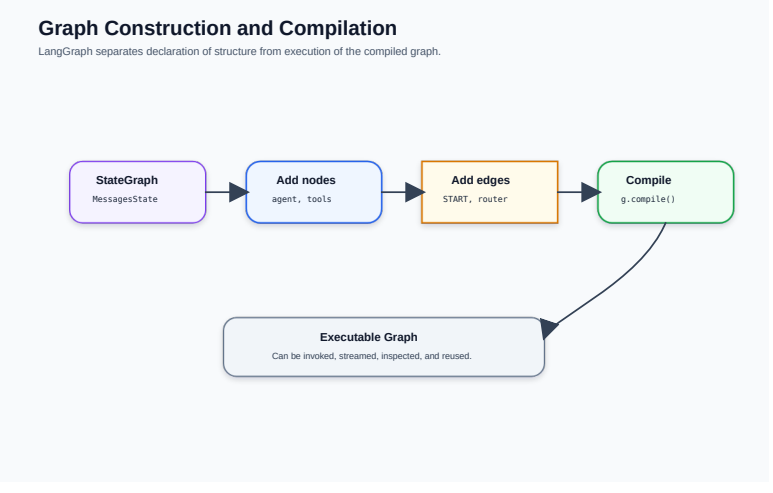
builder = StateGraph(MessagesState)
builder.add_node("agent", agent_node)
builder.add_node("tools", tool_node)
builder.add_edge(START, "agent")
builder.add_conditional_edges("agent", router)
builder.add_edge("tools", "agent")   # el ciclo
graph = builder.compile()

out = graph.invoke(
    {"messages": [HumanMessage(q)]},
    config={"recursion_limit": 10})
```



recursion_limit (default 25) cuenta pasos del grafo; al superarlo se lanza GraphRecursionError.

Compilar, ejecutar e inspeccionar




```python
# delta de cada nodo, en orden
for update in graph.stream(
        {"messages": [HumanMessage(q)]},
        stream_mode="updates"):
    for node, delta in update.items():
        print(node, "→", delta["messages"][-1])

# dibujar el grafo
graph.get_graph().draw_mermaid_png()
```

`compile()` valida la estructura (nodos referenciados existen, hay camino desde START) y devuelve un objeto inmutable con `.invoke()`, `.stream()` y `.get_graph()`.

Salida típica: `agent → tools → agent → tools → agent → END`.

Equivalencias: loop manual ↔ LangGraph

Loop manual (Parte 2)	LangGraph
lista `messages` + `append`	`MessagesState` + reducer `add_messages`
`client.chat.completions.create(...)`	`agent_node`
`for tc in tool_calls: fn(**args)`	`ToolNode(TOOLS)`
`if not msg.tool_calls: return`	`router` + `add_conditional_edges`
`continue` del `for`	arista `tools → agent`
`range(max_iter)`	`config={"recursion_limit": …}`
JSON schema escrito a mano + `REGISTRY`	`@tool` (schema desde la firma y el docstring)
`print` en cada paso	`graph.stream(stream_mode="updates")`

Mismo algoritmo, mismas herramientas, mismo modelo. Cambia dónde vive el control.

Resumen

- Un **agente con LLM** es el agente de la Clase 1 con π_θ = un LLM: $a_t \sim \pi_\theta(\cdot \mid h_t)$, $o_t = E(a_t)$, $h_{t+1} = h_t \oplus a_t \oplus o_t$.
- El **loop agéntico** es código Python ordinario: llamar al LLM, si hay `tool_calls` ejecutarlas y agregar observaciones, si no, responder. $T_{\max}$ es obligatorio.
- El LLM es **stateless**: la memoria es la lista de mensajes. El costo de un episodio crece como $O(T^2)$ en tokens.
- **CoT** marginaliza sobre razonamientos (self-consistency la estima por votación); **ReAct** intercala pensamientos y acciones; **ToT** convierte el razonamiento en búsqueda con heurística.
- Una **herramienta** tiene dos caras: esquema JSON (lo que ve el modelo) y función (lo que ejecuta el runtime). La tool call es generación estructurada: hay que validarla.
- **Extensiones** (ejecuta el agente), **funciones** (ejecuta el cliente) y **almacenes de datos** (RAG como herramienta).
- **LangGraph** expresa el mismo loop como grafo $(V, E, S, \oplus)$: nodos que devuelven actualizaciones parciales, un reducer, y un router condicional.

EAFIT

UNIVERSIDAD

Fin de la Clase 12

Próxima clase: Sistemas multi-agente